

VectorWeaver: A Stage-Aware Automated Optimization Framework for LLM Inference on Edge Platform

PENGFEI YANG, Xidian University, Xi'an, China

HUI ZENG, Xidian University, Xi'an, China and Zhongguancun Academy, Beijing, China

WENXUAN HOU, Xidian University, Xi'an, China

MINGWEI WANG, Xidian University, Xi'an, China

WEIYE JI, Xidian University, Xi'an, China

TIANYANG ZHENG, Xidian University, Xi'an, China

HUI LI, Xidian University, Xi'an, China

The convergence of Large Language Models (LLMs) and the open-source RISC-V architecture creates significant opportunities for specialized AI hardware. However, achieving optimal LLM inference performance on RISC-V platforms is challenging due to the complex computational patterns of LLM operators and the architectural nuances of the RISC-V Vector (RVV) extension. Existing optimization strategies often rely on manual, expert-driven tuning, which is labor-intensive and not generalizable across diverse operators and hardware configurations. This article presents VectorWeaver, an automated framework to generate high-performance, stage-aware microkernels for LLM inference on RISC-V. VectorWeaver specifically addresses key performance bottlenecks in quantized models, such as memory discontinuities in vector dot-product operations, by employing a widen vectorization strategy. Guided by a stage-aware performance model, the framework systematically explores a rich optimization space encompassing loop unrolling, software pipelining, and mathematical approximations. The model generates distinct, highly-optimized kernels for the prefill and decode stages, selected at runtime via a dispatch mechanism. We evaluate VectorWeaver on the TH1520 and the Sophon SG2044 using Gemma3 and Qwen3 models with sizes ranging from 270M to 4B, where our generated microkernels achieve up to a 24x speedup over baseline implementations. For end-to-end evaluations, VectorWeaver delivers up to a 5.2x increase in inference throughput, demonstrating its effectiveness in automating the generation of high-performance code and advancing the state of LLM inference on the RISC-V ecosystem.

CCS Concepts: • **Computing methodologies** → *Vector / streaming algorithms*;

Additional Key Words and Phrases: Vector extension, RISC-V, large language model

This work was supported in part by the Shaanxi Key Technology R&D Program under Grant 2024GX-ZDCYL-02-15, in part by the Natural Science Funds for Distinguished Young Scholar of Shaanxi under Grant 2025JC-JCQN-079. This work is also supported by the Zhongguancun Academy, (Grant No. C20250501).

Authors' Contact Information: Pengfei Yang, Xidian University, Xi'an, Shaanxi, China; e-mail: pfyang@xidian.edu.cn; Hui Zeng (corresponding author), Xidian University, Xi'an, Shaanxi, China and Zhongguancun Academy, Beijing, China; e-mail: hzeng@stu.xidian.edu.cn; Wenxuan Hou, Xidian University, Xi'an, Shaanxi, China; e-mail: wxhou@stu.xidian.edu.cn; Mingwei Wang, Xidian University, Xi'an, Shaanxi, China; e-mail: 2669058631@qq.com; Weiye Ji, Xidian University, Xi'an, Shaanxi, China; e-mail: weyeeji@qq.com; Tianyang Zheng, Xidian University, Xi'an, Shaanxi, China; e-mail: 503491300@qq.com; Hui Li, Xidian University, Xi'an, Shaanxi, China; e-mail: gray1y@stu.xidian.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 1544-3973/2026/03-ART35

<https://doi.org/10.1145/3799719>

ACM Reference Format:

Pengfei Yang, Hui Zeng, Wenxuan Hou, Mingwei Wang, Weiye Ji, Tianyang Zheng, and Hui Li. 2026. VectorWeaver: A Stage-Aware Automated Optimization Framework for LLM Inference on Edge Platform. *ACM Trans. Arch. Code Optim.* 23, 1, Article 35 (March 2026), 24 pages. <https://doi.org/10.1145/3799719>

1 Introduction

Large Language Models (LLMs) have become a dominant force in modern computing [35], yet their massive computational and memory requirements pose significant challenges for deployment on resource-constrained edge devices [46]. This challenge marks a critical inflection point in the evolution of artificial intelligence, often termed the shift from “Cloud AI” to “Edge AI” [41]. This paradigm shift is driven by burgeoning demands for real-time responsiveness, enhanced data privacy, and reduced operational costs, all of which are difficult to achieve when relying solely on remote data centers. The proliferation of intelligent applications in autonomous vehicles, smart homes, and personal robotics further necessitates powerful, on-device processing capabilities [26].

Concurrently, the RISC-V architecture is gaining momentum as a flexible, open-standard alternative to proprietary CPUs [34], particularly in the embedded and custom hardware space. Its modular, extensible nature makes it an ideal substrate for creating **domain-specific accelerators (DSAs)** tailored for AI workloads. The **RISC-V Vector (RVV)** extension, in particular, provides a powerful and adaptable mechanism for data-parallel computation, making it a promising target for accelerating the dense linear algebra operations at the heart of LLM workloads [5].

However, bridging the gap between high-level LLM frameworks like llama.cpp [16] and high-performance RVV assembly is non-trivial [18]. Achieving optimal performance is not merely a matter of straightforward code compilation; it is a multi-faceted problem fraught with challenges that lie at the intersection of algorithm, software, and hardware architecture [17].

We identify four primary hurdles that complicate this landscape. First, the ecosystem suffers from architectural fragmentation due to the intentionally flexible RVV specification, which allows implementers to choose different vector lengths (VLEN) and pipeline designs [11]. Consequently, an optimization strategy effective on one core may be suboptimal on another, demanding a portable methodology. Second, this hardware variability is compounded by operator complexity. The performance of key operators, particularly quantized matrix multiplication and activation functions, is highly sensitive to data layout and instruction scheduling [1], where subtle issues like memory discontinuities in quantization schemes can cripple naive vectorization. Third, the workload itself exhibits a prefill-decode dichotomy. LLM inference is not monolithic but comprises two distinct phases: the parallel, compute-bound “Prefill” stage and the sequential, memory-bound “Decode” stage [21, 48]. A truly effective framework must be “stage-aware,” as a single, statically optimized kernel is unlikely to suffice for both. Finally, these challenges collectively create a manual optimization bottleneck. Current state-of-the-art solutions rely on labor-intensive, expert-driven tuning that is inherently fragile—failing to scale to new operators, evolving architectures, or next-generation hardware.

To address these challenges, we propose VectorWeaver, an automated framework designed to generate highly optimized, stage-aware microkernels for LLM inference on RISC-V platforms. VectorWeaver automates the entire optimization workflow, from high-level operator definitions to the synthesis of fine-tuned RVV assembly code, making principled, architecture-aware decisions to navigate the complex optimization space.

Our contributions are threefold:

- **Novel RISC-V Microkernel Optimizations:** We introduce a “Width Expansion” strategy that reorganizes in-register data to overcome memory access bottlenecks in quantized

dot-product kernels, significantly improving SIMD efficiency. This is combined with systematic loop unrolling and mathematical approximations to maximize instruction-level parallelism and arithmetic throughput.

- **Automated and Stage-Aware Framework:** We design and implement VectorWeaver, a fully automated framework that translates high-level kernel definitions into a novel **Vector-Adaptive Intermediate Representation (VA-IR)**. It then explores a diverse space of optimization strategies, using a rank-based XGBoost performance model to identify the optimal kernel configurations for both the Prefill and Decode stages of LLM inference.
- **Comprehensive Evaluation:** We demonstrate the effectiveness of our approach through extensive experiments on real RISC-V hardware. VectorWeaver achieves significant speedups on critical micro-benchmarks (up to 24x) and delivers substantial end-to-end performance improvements (up to 5.2x) for llama.cpp, validating its capability to automate and significantly enhance LLM acceleration in the RISC-V ecosystem.

2 Background and Motivation

2.1 RISC-V Vector Extension (RVV)

The RVV extension enhances RISC-V with a flexible and powerful SIMD paradigm. Unlike more rigid SIMD architectures, RVV is designed to be highly configurable [19]. Its key concepts include:

- **Vector Registers:** A set of 32 vector registers ($v0-v31$). The physical length of these registers, $VLEN$, is implementation-defined (e.g., 128, 256, 512 bits).
- **Dynamic Configuration:** The `vsetvli` instruction is central to RVV's flexibility. It configures the vector unit at runtime, setting the standard element width (SEW) and the vector register grouping multiplier (LMUL). SEW can be 8, 16, 32, or 64 bits. LMUL (1, 2, 4, 8) allows multiple physical vector registers to be aggregated into a single, wider logical vector register. This provides a powerful mechanism for fine-grained control over data layout in registers, a capability we exploit in our work.
- **Masking and Predication:** Most vector instructions can be masked, allowing operations to be selectively applied to elements based on the value in a special mask register ($v0$), enabling efficient handling of conditional logic and irregular data structures without branching.
- **Memory Access:** RVV supports a rich set of memory access patterns, including unit-stride (ideal for contiguous arrays), strided (for accessing columns in a row-major matrix), and indexed (for gather/scatter operations). This versatility is essential for complex algorithms but also complicates the task of optimization.

This inherent flexibility is a double-edged sword: while it offers immense potential for performance tuning, it also creates a vast optimization space that is challenging for traditional compilers to navigate effectively. Manual optimization requires deep expertise in both the algorithm and the specific RVV implementation.

2.2 LLM Inference Characteristics: Prefill vs. Decode

LLM inference is a biphasic process with starkly different performance characteristics [3, 7, 25], as illustrated in Figure 1.

- **Prefill:** The initial prompt (of length S) is processed in a single, large batch. This phase is dominated by large matrix-matrix multiplications (e.g., query-key, key-value attention computations). The high data parallelism makes it compute-bound, where the performance is limited by the raw arithmetic throughput of the processor.
- **Decode:** New tokens are generated auto-regressively, one at a time. Each new token requires a full pass through the model's layers. This phase is characterized by matrix-vector

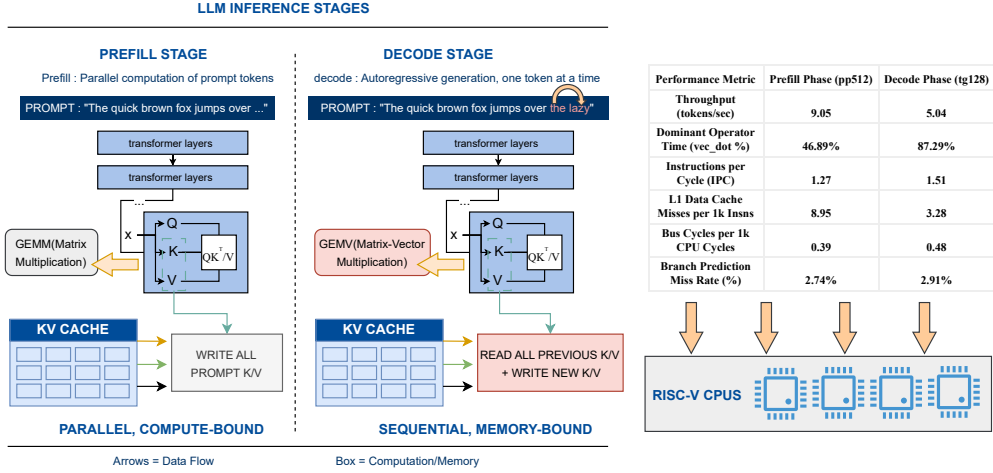


Fig. 1. Conceptual illustration of the two stages of LLM inference. The Prefill stage is a parallel, compute-bound operation, while the Decode stage is a sequential, memory-bound process.

multiplications, where the model weights (now including the KV cache) are streamed from memory. The low arithmetic intensity makes this phase memory-bound, where performance is limited by the latency and bandwidth of the memory subsystem.

This dichotomy implies that a single, statically optimized kernel is unlikely to be optimal for both phases. An aggressive, heavily unrolled kernel might excel in the prefill stage by maximizing ILP, but could suffer from instruction cache misses and increased register pressure in the memory-bound decode stage, leading to suboptimal performance.

2.3 Profiling and Bottlenecks in llama.cpp on RISC-V

To motivate our work, we profiled the widely-used “llama.cpp” framework on a RISC-V platform. A key bottleneck emerged in the quantized vector dot-product function (“ggml_vec_dot_q8_0”), which is fundamental to all matrix multiplications. The Q8_0 quantization format stores 32 8-bit integers (int8_t) along with a 16-bit floating-point scaling factor (ggml_half) for each block, as shown in Figure 2(a, b).

A direct vectorization of the dot product between two such blocks is inefficient. As illustrated in Figure 2(c), the 32 integer values can be loaded into a vector register, but the single scaling factor must be loaded separately into a scalar register and then broadcasted to another vector register before the final multiplication. This creates a long data dependency chain: the vector multiplication cannot begin until the scalar load and broadcast are complete. This serialization breaks the data flow, stalls the pipeline, and leads to a severe underutilization of the vector ALUs. The core issue is the **memory discontinuity** imposed by the quantization format, which prevents a simple, contiguous load of all necessary data for a fused multiply-add operation, thereby crippling SIMD throughput.

3 Microkernel and Optimization Strategies

To overcome the identified bottlenecks, we developed a suite of optimization strategies that form the core of VectorWeaver’s code generation capabilities. Our approach is founded on exploiting the unique flexibility of the RVV architecture.

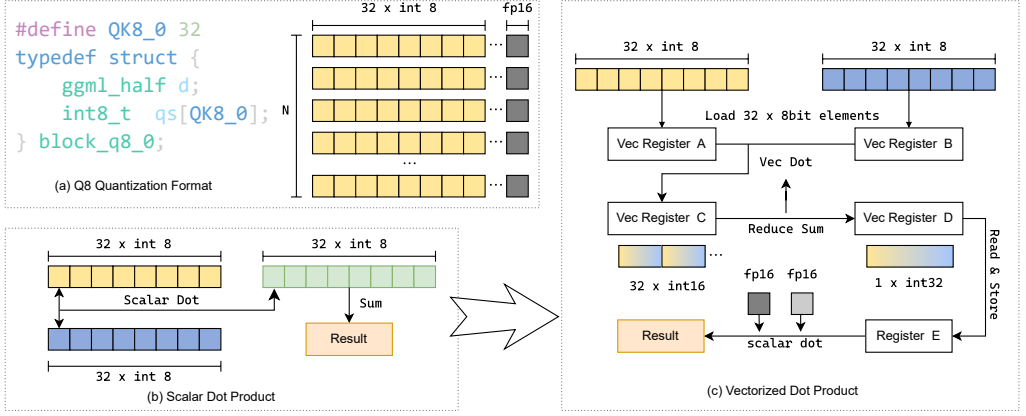


Fig. 2. Bottleneck in standard vectorization of Q8 dot product. (a) The Q8 quantization format interleaves a scalar scaling factor with a block of integers. (b) A scalar dot product is simple but slow. (c) A naive vectorized approach suffers from memory discontinuity, as the scaling factor must be loaded separately and broadcast, breaking the data flow and underutilizing vector ALUs.

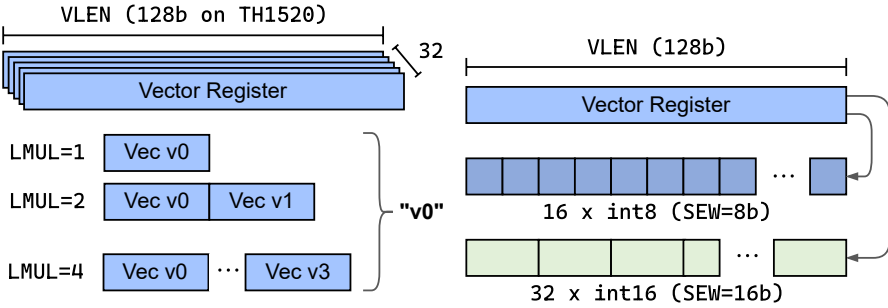


Fig. 3. The flexibility of RVV. The left side shows how LMUL can configure a group of physical vector registers (e.g., v_0, v_1) into a single logical vector register, enabling operations on wider data vectors. This provides an opportunity for fine-grained data orchestration.

3.1 Exploiting RVV's Flexible Data Layout

The core of our optimization philosophy lies in leveraging RVV's configurable vector registers. As shown in Figure 3, by adjusting the LMUL parameter via the `vsetvli` instruction, a programmer can treat a group of physical vector registers as a single, wider logical entity. For instance, with $VLEN=128b$ and $SEW=8b$, a standard vector register v_0 holds 16 elements. By setting $LMUL=2$, registers v_0 and v_1 are concatenated to form a logical register that can hold 32 8-bit elements. Setting $LMUL=4$ concatenates v_0-v_3 . This capability for fine-grained, in-register data arrangement provides a powerful tool to restructure data layouts post-load, enabling optimizations that are impossible in more rigid SIMD architectures.

3.2 Width Expansion for Quantized Kernels

We apply this principle to address the memory discontinuity issue in the “`vec_dot_q8`” kernel. We propose a “Width Expansion” strategy, detailed in Figure 4 and Algorithm 1. The key insight is to

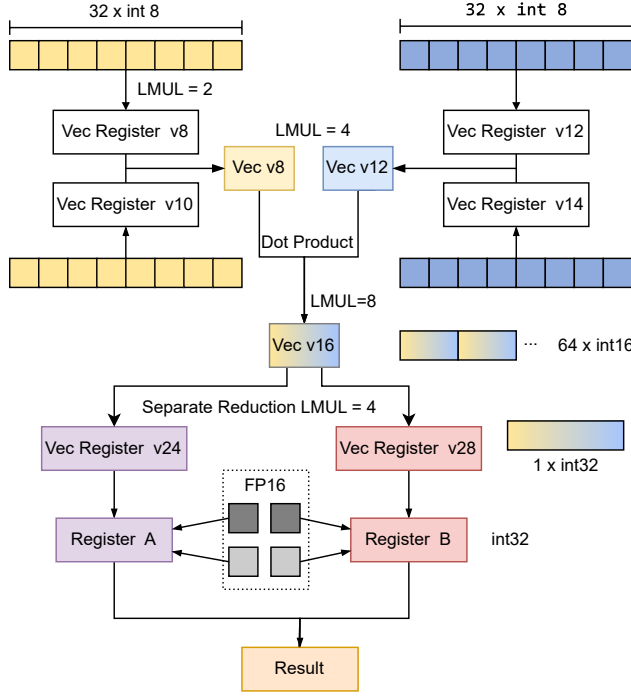


Fig. 4. The proposed “Width Expansion” strategy. By using a larger LMUL, we load multiple blocks of data and perform wider intermediate multiplications (e.g., 8-bit $\times$ 8-bit $\rightarrow$ 16-bit). This allows the scaling factors to be processed in parallel and better utilizes the full width of the vector registers, reducing loop overhead and improving arithmetic intensity.

restructure the computation to better align with the RVV’s architecture. Instead of performing a dot product on 8-bit integers and then scaling the final scalar result, we widen the operands within the vector registers before the main computation.

The process is as follows:

- (1) **Grouped Loading:** We set LMUL=4 or higher to load data for multiple Q8 blocks (e.g., two blocks of $32 \times \text{int}8$) into wide logical vector registers (Lines 9–12 in Algorithm 1).
- (2) **In-Register Widening:** We perform a widening vector-vector multiplication on the 8-bit integer data using `vwmul.vv`, producing 16-bit intermediate results which are stored in another wide vector register (Line 13).
- (3) **Parallel Scaling:** The corresponding scaling factors from each block are loaded and applied to the widened intermediate results in parallel (Lines 18–19). This is the crucial step that breaks the dependency chain seen in the naive approach.
- (4) **Efficient Reduction:** A final reduction sum is performed on the scaled, wider results.

This strategy transforms the problem from a series of narrow, dependent operations into a single, wider, and more parallel data flow. It maximizes the utilization of the vector ALU, reduces loop overhead, and better tolerates memory latency. However, Width Expansion is constrained by register availability: with 32 vector registers, the `vec_dot` kernel using full width (LMUL=m2) consumes ~ 16 physical registers per unrolled iteration, limiting achievable `unroll_factor` before triggering register spilling, which severely degrades performance. Split width (LMUL=m1) reduces

ALGORITHM 1: RVV Widening Implementation for “ggml_vec_dot_q8_0”

Require: $X, Y: n \times 32$ matrices, elements are `int8`
Require: $X_d, Y_d: n \times 1$ vectors, elements are `float16`
Ensure: $sumf$: cumulative result, type `float32`

```

1:  $num\_blocks \leftarrow n/32$ 
2:  $ib \leftarrow 0$ 
3:  $sumf \leftarrow 0.0$ 
4: for  $ib = 0$  to  $num\_blocks - 1$  step 2 do
5:    $sumi0 \leftarrow 0$ 
6:    $sumi1 \leftarrow 0$ 
7:    $v8 \leftarrow \text{load\_vl32\_e8m2}(X[ib])$ 
8:    $v10 \leftarrow \text{load\_vl32\_e8m2}(X[ib + 1])$ 
9:    $v12 \leftarrow \text{load\_vl32\_e8m2}(Y[ib])$ 
10:   $v14 \leftarrow \text{load\_vl32\_e8m2}(Y[ib + 1])$ 
11:   $v16 \leftarrow \text{widen\_mul\_vl64\_e8m4}(v8, v12)$ 
12:   $v24 \leftarrow \text{widen\_sum\_vl32\_e16m4}(v16)$ 
13:   $v28 \leftarrow \text{widen\_sum\_vl32\_e16m4}(v20)$ 
14:   $sumi0 \leftarrow v24$ 
15:   $sumi1 \leftarrow v28$ 
16:   $sumf \leftarrow sumf + sumi0 \times X_d[ib] \times Y_d[ib]$ 
17:   $sumf \leftarrow sumf + sumi1 \times X_d[ib + 1] \times Y_d[ib + 1]$ 
18: end for
19: return  $sumf$ 

```

per-iteration register consumption to ~ 9 registers, enabling higher unrolling degrees without spilling.

3.3 Loop Unrolling and Software Pipelining

To maximize **Instruction-Level Parallelism (ILP)** [37], VectorWeaver systematically applies loop unrolling. By increasing the number of independent instructions in the loop body, we provide the processor’s pipeline with more opportunities for parallel execution [4], which is critical for hiding the latency of both floating-point operations and memory accesses. This is particularly effective on superscalar RISC-V cores.

VectorWeaver implements this using inline RVV assembly, giving it fine-grained control over instruction scheduling. It parameterizes the unroll factor and generates code that explicitly interleaves load, compute, and store instructions from different logical loop iterations. This manual software pipelining is crucial for achieving performance close to the hardware’s peak, as it can explicitly manage data dependencies and prevent pipeline stalls from memory operations, a task where compilers often struggle [39]. The tradeoff is increased code size and register pressure, which our performance model must account for.

3.4 Mathematical Approximation

Certain activation functions, like SiLU (Sigmoid-weighted Linear Unit), are computationally expensive. The function, defined as $\text{SiLU}(x) = x \cdot \sigma(x) = x/(1 + e^{-x})$, involves the exponential function e^x , which is not a native RVV instruction and typically requires a slow, non-vectorized library call.

VectorWeaver incorporates a strategy for mathematical approximation to overcome this bottleneck. Instead of relying on generic library routines, it expands its search space to include fast, vectorized approximations. For the exponential function, one highly effective technique is a hardware-friendly variant based on direct manipulation of the IEEE 754 floating-point

representation [20]. This method leverages the insight that the integer representation of a floating-point number is approximately linearly proportional to its logarithm, allowing the exponential to be calculated using simple arithmetic operations.

The core of the approximation is formulated as follows. First, the input x is clamped to a valid range $[-C, C]$ to prevent overflow:

$$x' = \max(\min(x, C), -C). \quad (1)$$

Then, the exponential is approximated by scaling and shifting the clamped input and reinterpreting the result as a floating-point number:

$$e^{x'} \approx \text{reinterpret_cast} < \text{float} > (\text{round}(A \cdot x') + B), \quad (2)$$

where $A = 2^{23} / \ln(2)$ is a magic constant that scales the input to the 23-bit mantissa of a single-precision float while converting from base- e to base-2, and $B = (127 \ll 23)$ is a constant that applies the necessary exponent bias. This entire sequence—clamping, fused multiply-add, and rounding—can be implemented using a few highly efficient RVV instructions, completely avoiding the overhead of a library call.

This approximation introduces a tradeoff between performance and numerical precision. VectorWeaver treats this specialized implementation as another candidate kernel within its search space. It allows this strategy to be selectively enabled, providing a “knob” to tune for either maximum speed or full model accuracy. Our evaluation in Section 5 quantifies this tradeoff, showing that for many models, a slight approximation yields significant speedup with negligible impact on output perplexity.

These strategies—Width Expansion, Loop Unrolling, and Mathematical Approximation—are not mutually exclusive. VectorWeaver’s core strength lies in its ability to combine them. For a given operator, it can generate multiple candidate kernels, each representing a different combination of these techniques (e.g., a width-expanded kernel with an unroll factor of 4). The framework’s role, described next, is to automate the search for the best combination.

4 VectorWeaver: Automated RISC-V Microkernel Optimization

4.1 Automated Optimization Workflow

The optimization of LLM kernels on modern RISC-V platforms presents a formidable challenge, characterized by a vast and complex optimization space that spans instruction selection, data layout, and loop transformations. Manual optimization in this domain is not only a labor-intensive endeavor requiring deep architectural expertise but is also inherently fragile, as solutions are tightly coupled to a specific hardware implementation. VectorWeaver is conceived as a systematic solution to this problem, replacing ad-hoc, expert-driven tuning with a principled, automated workflow that robustly navigates this complexity. The framework’s workflow, depicted in Figure 5, orchestrates a seamless progression from a high-level, abstract kernel definition to the synthesis of multiple, highly-specialized, and hardware-specific assembly codes.

The process is initiated in the **IR Generation** stage. Here, a user-provided kernel, expressed in a high-level C-like syntax, is lowered into our custom VA-IR. This initial VA-IR instance captures the pure, unoptimized semantics of the operation. Subsequently, this representation is passed to the **Adaptive Strategy Exploration** stage, the core of the framework. The Weaver-Optimizer engine systematically applies a library of predefined transformation strategies to the VA-IR. This is not a single-pass transformation; instead, it generates a diverse population of distinct VA-IR instances, where each instance is a unique candidate kernel defined by a specific combination of optimizations (e.g., Width Expansion with an unroll factor of 4, versus a naive approach with an unroll factor of 2). This population of candidates is then fed into the **Performance Model Feedback** stage. This stage

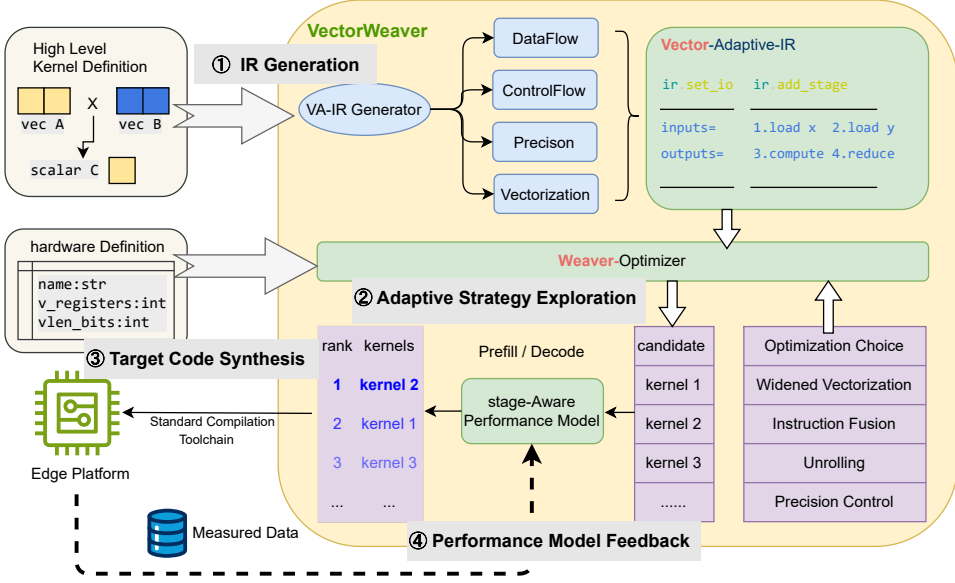


Fig. 5. The VectorWeaver Framework Workflow. A high-level kernel definition is first translated into our Vector-Adaptive IR (VA-IR). The Weaver-Optimizer then explores a space of optimization strategies to generate candidate kernels. These candidates are ranked for both Prefill and Decode stages by a stage-aware performance model, which is trained on measured hardware data. Finally, the top-ranked kernels for each stage are synthesized into target-specific assembly code.

employs a sophisticated machine learning model to efficiently predict the performance rank of each candidate, obviating the need for prohibitively slow, exhaustive on-hardware benchmarking. Finally, armed with these rankings, the **Target Code Synthesis** stage selects the optimal kernel for each required context (Prefill and Decode) and synthesizes it into low-level, highly-optimized RVV inline assembly, ready for direct integration into the target application.

4.2 Core Components: VA-IR and Performance-Guided Search

The efficacy and automation capabilities of the VectorWeaver framework are underpinned by two core technical components: its domain-specific intermediate representation designed for architectural modeling, and its intelligent, search-guiding performance model.

The Vector-Adaptive Intermediate Representation (VA-IR) serves as the central data structure and the primary abstraction layer within the framework. Its design is a direct response to the limitations of general-purpose compiler IRs like LLVM IR [42, 45], which are fundamentally target-agnostic. The VA-IR, by contrast, is intentionally target-aware, providing a parametric model of the RVV architecture’s configurable features. Unlike SSA-based representations that focus on data flow analysis, VA-IR adopts a kernel-centric design that explicitly decouples computation semantics from optimization decisions, enabling the systematic exploration of the RVV optimization space.

A VA-IR program is structured around three core abstractions. The **KernelIR** component defines the overall kernel structure, including loop bounds, input/output ports with type annotations, and a sequence of named computation stages. Each stage encapsulates a logically coherent operation phase

within the loop body, such as data loading, arithmetic computation, or reduction. Between stages, VA-IR supports embedding C++ code fragments—termed *interludes*—to handle non-vectorizable operations like scale factor computation in quantized kernels. This multi-stage design naturally captures the structure of complex LLM operators that interleave vector and scalar operations.

Within each stage, the computation is expressed through **IROperations**—primitive vector operations that abstract RVV instruction semantics. These operations span five categories: memory operations (load, store) for vector data movement; data extraction operations (unpack_lo, unpack_hi) for nibble and byte manipulation in quantized formats; compute operations (vwmul, vwmacc, vfmul) covering widening multiply, fused multiply-accumulate, and floating-point arithmetic; reduction operations (vwredsum, vfredosum) for vector-to-scalar aggregation; and conversion operations (vfcvt, vnsrl) for type transformations. Crucially, IROperations do not prescribe specific register allocations or instruction encodings; instead, they declare the semantic intent, allowing the code generator to make target-specific decisions during synthesis.

The third abstraction, **TuningConfig**, encodes the optimization parameters that collectively define the search space for auto-tuning. These parameters are organized into four categories that reflect distinct optimization concerns. *Code generation level* determines whether the output targets portable RVV intrinsics or hand-optimized inline assembly for maximum control. *Loop optimization* parameters include unroll factor, which specifies how many loop iterations are processed together; pipeline style, which controls whether unrolled iterations execute in a serial load-compute-store pattern or an interleaved pattern that overlaps memory and compute operations; and block batch size for processing multiple quantization blocks per iteration. *RVV-specific parameters* capture architectural decisions unique to vector processors: LMUL strategy selects between full width (e.g., LMUL=m2 processing 32 elements) or split mode (LMUL=m1 processing 2×16 elements with explicit halving); instruction fusion enables combining separate widening multiply and add operations into a single vwmacc instruction; and widening mode controls the use of widening operations for precision preservation. Finally, *precision and approximation control* governs numerical tradeoffs: accumulator strategy selects f32 or f64 precision for numerical stability in reduction-heavy kernels, while mathematical approximation options enable fast implementations of transcendental functions.

The key insight underlying VA-IR’s design is that the same kernel description, when combined with different TuningConfig instantiations, yields fundamentally different code variants—each optimized for a specific hardware context or execution phase. This transforms the optimization task from opaque code generation into a well-defined search problem over the VA-IR’s parameter space, effectively enabling the optimizer to perform a “what-if” analysis for myriad optimization combinations before any code is generated. Figure 6 illustrates this mapping: the IROperations in a kernel definition (Figure 6(a)) are lowered to RVV assembly sequences (Figure 6(b)) through template instantiation, with the specific code structure determined by the TuningConfig parameters (Figure 6(c)).

The Performance-Guided Search intelligently navigates this vast parameter space using a stage-aware machine learning model. We employ an XGBoost model specifically configured for a *learning-to-rank* task, a paradigm chosen for its robustness against the inherent noise and variability of physical hardware measurements. A model that predicts relative performance ranks is less susceptible to being misled by transient system effects than one predicting absolute latency. The model’s predictive power is derived from a meticulous feature engineering process. We extract a rich feature vector from each VA-IR candidate, including: (1) *Static Code Features* like the ratio of ALU operations to memory accesses, the complexity of loop-carried dependencies, the degree of ILP, and an analytical estimate of register pressure; and (2) *Contextual Features* such as the input sequence length and a crucial binary flag, `is_prefill`. The model is trained offline on a dataset collected by benchmarking a curated, representative subset of kernel configurations on the

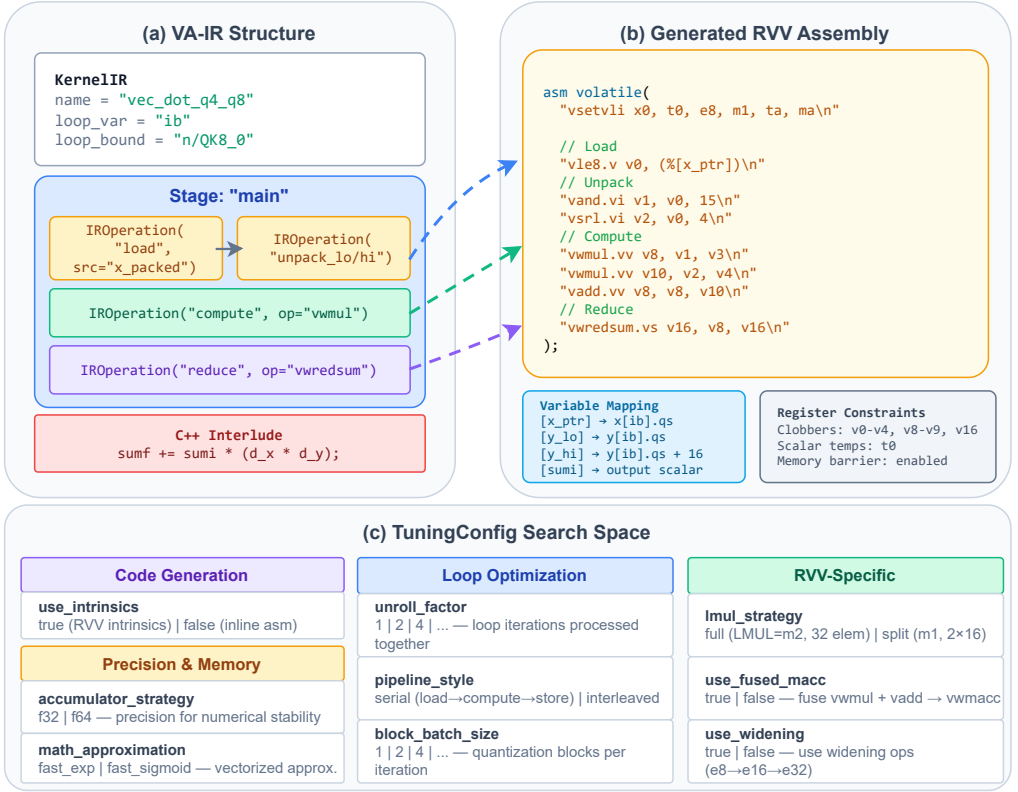


Fig. 6. VA-IR representation and code generation workflow. (a) Kernel structure with IROperation primitives and C++ interludes. (b) Generated RVV assembly with variable mapping and register constraints. (c) TuningConfig search space spanning code generation, loop optimization, precision control, and RVV-specific parameters.

target hardware, learning the complex, non-linear relationships between these features and the final performance. This enables the model to accurately predict the best-performing kernels for unseen combinations and different execution contexts. The search space is explored via offline grid search with heuristic pruning. Critically, the model receives *hardware parameters*—including VLEN, register count, and cache sizes—as contextual features alongside TuningConfig parameters. These hardware features enable the pruning rules to filter configurations exceeding platform-specific constraints (e.g., configurations requiring >32 registers are rejected). XGBoost is trained on SG2044 [6] measurements and achieves 75% accuracy when transferred to TH1520 without retraining, demonstrating adaptability. This optimization process typically completes within 1 to 3 minutes for a target kernel. The cost model inference itself requires less than 20 seconds to rank the search space, with the remaining time dedicated to VA-IR code generation and compilation checks. Consequently, the total one-time offline tuning cost is amortized across all subsequent inference workloads.

4.3 Stage-Aware Kernel Specialization and Dispatch

A cornerstone innovation of VectorWeaver is its automated handling of the biphasic performance profile of LLM inference. While the Prefill stage is compute-bound and throughput-oriented, the

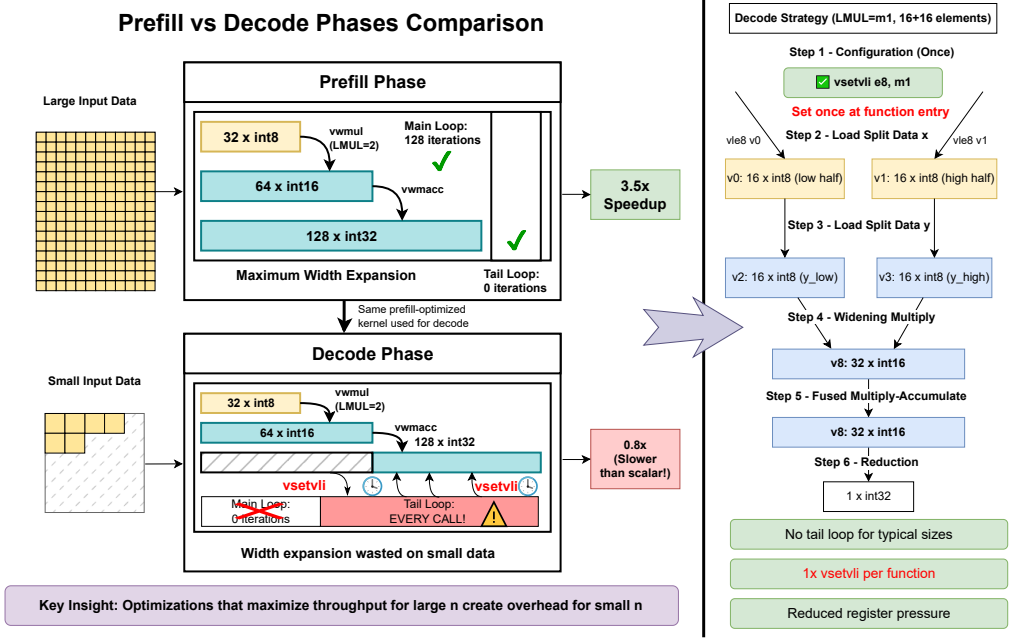


Fig. 7. Comparison of optimization strategies for Prefill versus Decode phases. The Prefill strategy (top) maximizes throughput via Width Expansion and loop unrolling, absorbing overhead through large data volumes. Conversely, applying this strategy to the Decode phase (bottom left) incurs severe penalties due to frequent `vsetvli` transitions and tail loop dominance. The specialized Decode strategy (right) optimizes for latency by fixing LMUL=1, splitting data loads to fit physical registers, and eliminating dynamic configuration overhead.

Decode stage is memory-bound and latency-sensitive. Our experiments on the SG2044 platform reveal that this distinction is not merely a matter of tuning parameters but requires fundamentally divergent architectural approaches. As illustrated in Figure 7, blindly applying throughput-oriented optimizations to the Decode phase can lead to catastrophic performance degradation.

We identify three primary sources of overhead that make Prefill-optimized kernels detrimental to the Decode stage, particularly on newer RISC-V implementations like the SG2044:

- **Configuration Latency (`vsetvli`):** Prefill kernels, such as those employing our Width Expansion strategy, frequently toggle the vector register grouping (LMUL) to maximize **Arithmetic Logic Unit (ALU)** utilization. In the Decode stage, where the problem size is small (often a single token), the cycle cost of these `vsetvli` instructions becomes a significant percentage of the total execution time, negating any arithmetic gains.
- **Tail Loop Dominance:** Aggressive widening and unrolling require large multiples of data elements. When processing small batches in the Decode stage, the majority of the data often falls into the scalar “tail loop” rather than the vectorized main loop, forcing the CPU into a slower execution path.
- **Granularity Mismatch:** On hardware where the physical vector length (VLEN) is limited (e.g., 128 bits), a standard quantization block ($32 \times \text{int8}$) exceeds a single register’s capacity. While Prefill kernels can mask the cost of loading and widening this data across many iterations, Decode kernels suffer from the overhead of managing these splits dynamically.

To address these challenges, VectorWeaver employs a dual-surface learning approach. During synthesis, the performance model is queried with distinct context flags (`is_prefill`) for each operator. For the **Prefill stage**, the model selects kernels utilizing Maximum Width Expansion and deep software pipelining (Figure 7, top), as the massive data volume successfully amortizes the instruction overhead. However, for the **Decode stage**, the framework shifts to a “latency-minimalist” strategy (Figure 7, right). Instead of widening vectors dynamically, VectorWeaver generates kernels that maintain a static configuration (typically $LMUL=1$) throughout the function’s lifespan. This involves techniques such as splitting input data across multiple registers during the load phase to fit fixed register widths, thereby eliminating the need for mid-kernel `vsetvli` reconfigurations and minimizing register pressure. This approach also reduces the overhead often introduced by standard intrinsics, favoring lean, inline assembly sequences.

This automated decision-making is materialized via a lightweight **runtime dispatcher**. Integrated into the inference loop, this dispatcher directs program flow based on token count with negligible overhead. This mechanism effectively resolves the “performance seesaw” observed in static optimization strategies—where optimizing for one stage penalizes the other. As we will demonstrate in Section 5 (specifically the ablation study in Figure 7), VectorWeaver’s ability to dispatch the optimal kernel for each phase allows it to match the peak throughput of specialized static baselines (PP-Best and TG-Best) simultaneously, without compromising either.

5 Evaluation

We conducted a series of experiments to evaluate the effectiveness of VectorWeaver, focusing on both micro-benchmark performance and end-to-end application speedup.

5.1 Experimental Setup

Hardware and Software Platform: The experiments were conducted on two distinct RISC-V platforms: a LicheePi 4A development board and a Sophon SG2044 platform. The specific configurations are as follows:

- **Hardware:** The LicheePi 4A is equipped with a T-Head TH1520 SoC, featuring a 4-core C910 CPU running at 1.85 GHz and supporting the RVV 0.7.1 extension, with 16 GB of RAM. The SG2044 platform is equipped with a Sophon SG2044 SoC, featuring a 64-core C920v2 CPU running at 2.6 GHz and supporting the RVV v1.0.0 extension.
- **Software:** The TH1520 platform runs the openKylin 2.0 SP1 operating system with Linux kernel 5.10.113. The SG2044 platform runs the openEuler 24.03 (LTS) operating system with Linux kernel 6.12.56. All C++ code was compiled using the RISC-V GCC toolchain.

Micro-benchmarks: To evaluate the performance of individual operators, we conducted micro-benchmarks comparing our generated kernels against several baselines:

- **Scalar C++ (O1):** A baseline C++ implementation without vectorization, compiled with `-O1` optimization.
- **Scalar C++ (O3):** The same scalar implementation compiled with the aggressive `-O3` optimization level to evaluate standard compiler effectiveness.
- **VectorWeaver (Intr):** A manually-tuned version using RVV intrinsics, representing a strong, expert-optimized baseline.
- **VectorWeaver (Best):** The best automatically discovered kernel by our framework, provided in both assembly-level and intrinsic-based implementations.

End-to-End Evaluation: To measure the impact on real-world application performance, we integrated our framework into the `llama.cpp` environment. We evaluated the end-to-end

throughput across a suite of models, including Qwen-3 (0.6B, 1.7B) and Gemma-3 (270M, 4B), to assess performance on various scales. The baselines for this evaluation are:

- **Ollama [36]**: A popular, packaged framework for serving LLMs, representing an easy-to-use, out-of-the-box performance baseline.
- **llama.cpp (Scalar)**: The official release of the `llama.cpp` project, a widely-recognized C++ framework for efficient LLM inference across a wide range of hardware.
- **VectorWeaver (Intr)**: A version generated by our framework where the search space is restricted to kernels implemented with standard RVV intrinsics. This baseline isolates the performance gains of our advanced assembly-level optimizations over a framework-managed, intrinsic-based approach.
- **VectorWeaver (Best)**: The full implementation of our VectorWeaver framework, which explores an expanded search space including advanced assembly-level optimizations and uses the stage-aware mechanism to deploy the best-found kernels for each inference phase. Notably, all kernels in this configuration maintain strict mathematical equivalence to the original operators

Ablation Study Configurations: To isolate the benefits of our stage-aware scheduling and validate the numerical fidelity of our kernel optimizations, we established three specific configurations for comparison:

- **PP-Best**: A version statically compiled to exclusively use the microkernels found to be optimal for the **Prompt Processing (Prefill)** stage for all operations.
- **TG-Best**: A version statically compiled to exclusively use the microkernels found to be optimal for the **Token Generation (Decode)** stage for all operations.
- **VectorWeaver+SiLU**: A variant utilizing IEEE 754-based approximations to enable fully vectorized SiLU. To evaluate the impact of this approximation on model fidelity, we utilize Perplexity (PPL) as the primary metric. PPL is widely adopted to measure LLM degradation under quantization or approximation [14, 43]. Specifically, we report the PPL on the WikiText-2 [33] benchmark using the llama-perplexity evaluation harness. A negligible increase in PPL indicates that the optimized operators maintain the numerical fidelity of the original model.

5.2 Micro-Benchmark Results

We evaluated the performance of individual kernels critical to LLM inference on two distinct RISC-V platforms: the LicheePi 4A (TH1520) and the Sophon SG2044. Table 1 presents the execution latency for key operators, covering vector dot products, row quantization, normalization, GEMM, and activation functions. We compared the code generated by VectorWeaver (in both Intrinsic and Assembly modes) against standard scalar C++ implementations (-O1) and compiler auto-vectorized versions (-O3).

The results demonstrate that VectorWeaver achieves substantial performance gains across all tested kernels on both platforms, with the generated assembly code consistently outperforming the baselines.

On the TH1520 platform, VectorWeaver achieves speedups ranging from 2.02x to 17.57x compared with the scalar baseline. For memory-bound operations like RMS Norm, our assembly generation reduces latency from 5.74 μ s to 0.65 μ s, representing an 8.73x speedup. The most significant improvement is observed in the Vec-SiLU operator. By combining automated SIMD optimization with mathematical approximation, VectorWeaver reduces the latency from 83.96 μ s to 4.77 μ s, achieving a 17.57x speedup.

The advantages of our framework are even more pronounced on the SG2044 platform. The Vec-Dot (Q8, Q8) kernel sees a 6.89x speedup, significantly higher than the 4.42x observed on

Table 1. Micro-Benchmark Performance of Key LLM Operators

Operator	Unit	Scalar C++		VectorWeaver		Speedup
		−O1	−O3	Intr.	Asm	
LicheePi 4A (TH1520)						
Vec-Dot (Q8, Q8)	μs	4.490	4.157	2.262	1.014	4.42x
Vec-Dot (Q4, Q8)	μs	4.707	2.834	2.254	1.103	4.26x
Quantize Row (Q8)	μs	22.195	22.049	5.654	4.722	4.70x
RMS Norm (F32)	μs	5.741	5.832	1.180	0.657	8.73x
GEMM (Q4, Q8)	ms	42.842	24.443	23.218	21.190	2.02x
Vec-SiLU (F32)	μs	83.960	83.760	4.781	4.776	17.57x
Sophon SG2044						
Vec-Dot (Q8, Q8)	μs	3.046	2.376	0.511	0.442	6.89x
Vec-Dot (Q4, Q8)	μs	3.132	2.260	1.625	0.508	6.17x
Quantize Row (Q8)	μs	18.082	14.800	4.196	3.254	5.56x
RMS Norm (F32)	μs	4.996	4.960	0.894	0.894	5.59x
GEMM (Q4, Q8)	ms	32.027	66.668	8.282	7.641	4.19x
Vec-SiLU (F32)	μs	61.626	61.579	4.462	2.483	24.82x

Comparison of latency between VectorWeaver (VW) and Scalar C++ baselines. Speedup denotes the improvement of VW (Asm) over the Scalar C++ (-O1) baseline.

the TH1520, indicating that our framework effectively exploits the wider vector width or superior memory bandwidth. The Vec-SiLU operator again shows the highest efficiency, with a remarkable 24.82x speedup. Furthermore, the GEMM (Q4, Q8) operator achieves a 4.19x speedup on SG2044, compared with 2.02x on TH1520, demonstrating better scalability for compute-intensive workloads.

The evaluation underscores the advantage of our hierarchical approach over standard compilation. While VectorWeaver’s intrinsic mode offers substantial gains, the assembly mode unlocks peak performance, exemplified by the SG2044 SiLU kernel where latency drops from 4.46 μs (Intr.) to 2.48 μs (Asm). In contrast, general-purpose compiler auto-vectorization (-O3) proves unreliable for these specialized quantized workloads. It occasionally causes severe regressions—most notably in the SG2044 GEMM operator, where -O3 optimization (66.67 ms) lags significantly behind even the scalar -O1 baseline (32.03 ms). This disparity validates the necessity of VectorWeaver’s domain-specific automated generation over reliance on generic compiler heuristics.

5.3 End-to-End Performance

To assess the real-world impact of our framework, we integrated VectorWeaver into the `llama.cpp` environment and measured the end-to-end inference performance. The evaluation spans two distinct phases: Prompt Processing (PP, 512 tokens) and Token Generation (TG, 128 tokens). We report results on both the TH1520 and SG2044 platforms across multiple model sizes (Qwen-3, Gemma-3) and quantization levels (Q4, Q8).

Figure 8 summarizes the normalized speedup of VectorWeaver against the baseline `llama.cpp` (Scalar) and Ollama. In the prefill phase, VectorWeaver achieves substantial acceleration by effectively utilizing RVV instructions. On the TH1520 platform, the fully optimized VectorWeaver (Best) achieves a **3.78× speedup** on Qwen-3 1.7B (Q8), increasing throughput from 1.07 to 4.05 tokens/s.

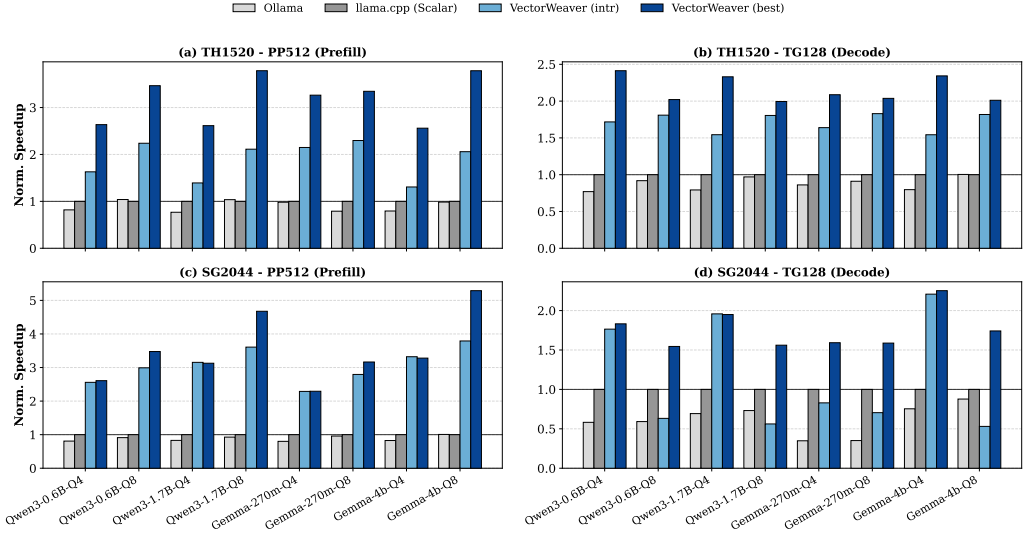


Fig. 8. Normalized end-to-end speedup over llama.cpp (Scalar). The comparison covers Prompt Processing (PP512) and Token Generation (TG128). VectorWeaver (Best) achieves up to **5.25×** speedup in the compute-bound PP phase. In the TG phase, it consistently outperforms the baseline, effectively recovering performance in cases where intrinsic-only optimizations regress (e.g., SG2044 Q8).

Similarly, for Gemma-3 270M (Q8), throughput improves by 3.35× (33.85 vs. 10.12 tokens/s). On the SG2044, the gains are even more pronounced due to higher parallelism. We observe a **5.25× speedup** for the Gemma-3 4B (Q8) model. The results demonstrate that our framework effectively eliminates the computation bottleneck in the prefill stage.

In the decoding phase, despite the memory bandwidth constraints, VectorWeaver maintains a competitive edge. On TH1520, we observe consistent improvements, such as a 2.33× speedup for Qwen-3 1.7B (Q4). A key finding appears on the SG2044 platform with Q8 quantization. As shown in Figure 8(d), manual or intrinsic-based optimizations (*VectorWeaver Intr*) occasionally suffer from performance degradation compared with the scalar baseline (e.g., Qwen-3 0.6B Q8 drops to 0.63× of baseline) due to inefficient compiler scheduling for complex vector instructions. However, *VectorWeaver (Best)* successfully overcomes this by automating the search for optimal kernel configurations, recovering the performance to a **1.54× speedup**. This validates the robustness of our automated search framework in diverse hardware-scenarios.

We further evaluated the scalability of VectorWeaver by varying the thread count (1-4 for TH1520, 16-64 for SG2044). Figure 9 correlates inference throughput with CPU utilization.

The results highlight VectorWeaver’s superior resource efficiency. The framework can match the peak performance of the baseline using significantly fewer computational resources:

- TH1520: For the Qwen-3 1.7B (Q8) prefill task, VectorWeaver achieves the baseline’s 4-thread throughput using only ~1 thread, representing a **74.5% reduction in CPU usage**.
- SG2044: Similarly, for Qwen-3 0.6B (Q8), VectorWeaver matches the baseline’s 64-thread performance with only ~16 threads, yielding a **74.2% CPU saving**.

Even in memory-bound scenarios like Gemma-3 270M (Q8) decoding on TH1520, our approach yields a **62.1% CPU saving** at iso-throughput. This efficiency is critical for edge deployments, allowing the saved computing power to be allocated to other concurrent system tasks.

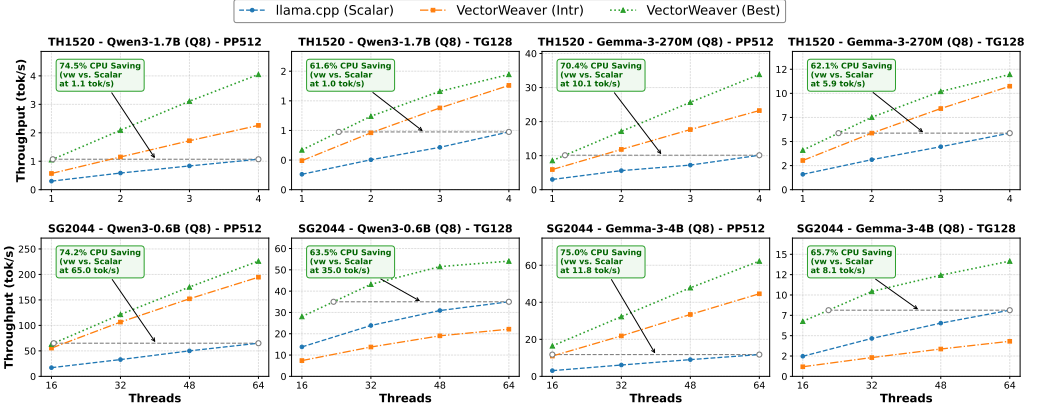


Fig. 9. Throughput scaling versus CPU utilization. The gray dashed line represents the baseline's peak performance using all cores. Green boxes highlight the **CPU savings** (up to 75.0%) achieved by VectorWeaver when matching the baseline's peak throughput, demonstrating superior efficiency on edge platforms.

5.4 Ablation Study on Stage-Aware Optimization

A central hypothesis of our work is that the divergent computational characteristics of the Prefill and Decode stages necessitate specialized microkernels. To quantitatively validate this, we conducted a targeted ablation study comparing our full dynamic framework against the static PP-Best and TG-Best configurations.

As illustrated in Figure 10, the experimental outcomes strongly corroborate our hypothesis, exposing the significant performance tradeoffs inherent in static “one-size-fits-all” optimization strategies. This tradeoff is particularly pronounced on the high-performance SG2044 platform. For instance, with the Qwen-0.6B model on SG2044: The PP-Best configuration achieves a high prefill throughput of 227.2 tok/s but suffers drastically in the decoding phase, only achieves 24.4 tok/s—significantly lower than TG-Best’s 54.1 tok/s due to the overhead of managing large tile sizes in a memory-bandwidth-limited context.

A similar trend is observed on the resource-constrained TH1520 platform. For the Gemma-4B model, TG-Best provides the optimal decoding speed (0.88 tok/s) but lags in prefill (1.46 tok/s) compared with PP-Best (1.93 tok/s). These disparities highlight that no single static kernel can optimally serve both phases.

VectorWeaver successfully navigates this dilemma through its stage-aware dispatch mechanism. By dynamically selecting the optimal kernel for each phase, it consistently matches the best-in-class performance for both stages. On the SG2044 (Qwen-0.6B), *VectorWeaver* achieves 226.1 tok/s in prefill and 54.1 tok/s in decoding, effectively mirroring the peak performance of PP-Best and TG-Best respectively, with negligible runtime overhead.

This ablation study provides empirical evidence that stage-aware optimization is essential for RISC-V LLM inference. The performance gap between static strategies widens as hardware parallelism increases (as seen on SG2044), making the dynamic dispatch mechanism of *VectorWeaver* not merely beneficial, but critical for maximizing hardware utilization across the entire inference lifecycle.

5.5 Ablation Study on Mathematical Approximation

In addition to stage-aware scheduling, *VectorWeaver* introduces mathematical approximation as a strategy to overcome the bottleneck of transcendental functions, specifically the SiLU activation

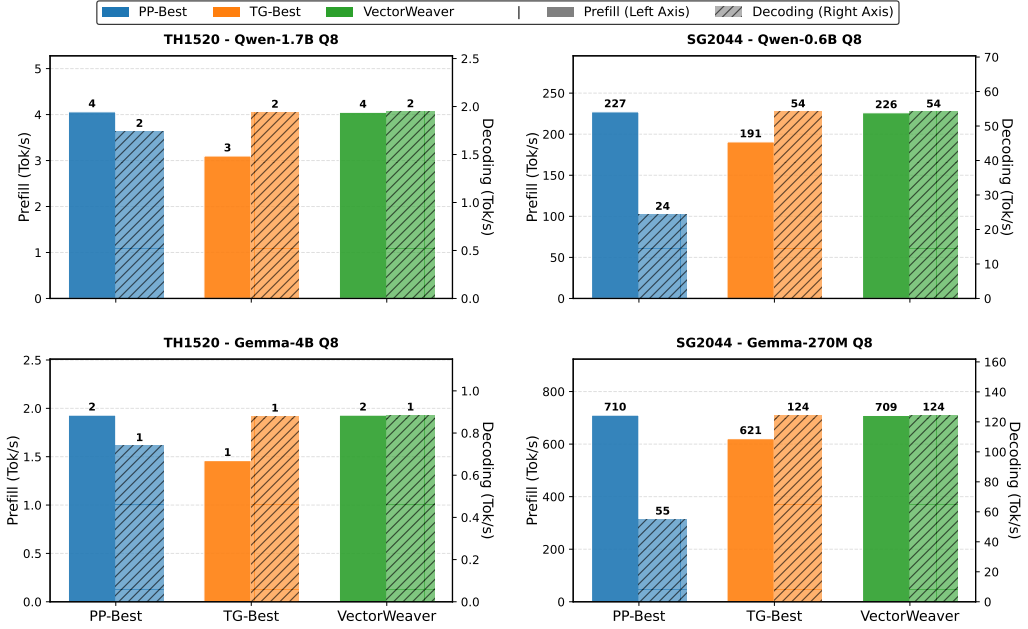


Fig. 10. Performance comparison of static optimization strategies (PP-Best, TG-Best) versus the dynamic VectorWeaver framework. The evaluation covers Qwen and Gemma model variants on both TH1520 and SG2044 platforms using Q8 quantization. Left Axis (Solid Bars): Prefill throughput; Right Axis (Hatched Bars): Decoding throughput.

used in the Qwen model family. To quantify the tradeoff between computational throughput and model fidelity, we evaluated the impact of our approximated SiLU kernel on both perplexity (PPL) and inference speed.

Table 2 details the comparison between the scalar baseline, our standard **VectorWeaver** implementation, and an approximated variant (**VectorWeaver+SiLU**). In line with the modular design of the framework, this approximation is provided as an optional configuration and can be disabled to guarantee zero precision loss, analogous to the choice between intrinsic-based and assembly-based backends. The results reveal a distinct performance characteristic: while the approximation yields marginal gains in the **Prompt Processing (PP)** stage compared with the exact vector kernel, it delivers significant speedups in the **Token Generation (TG)** stage.

For the Qwen3-0.6B (Q4_0) model, employing the approximated SiLU boosts the TG throughput from 64.45 tokens/s (exact vector) to 83.00 tokens/s, a notable 28.8% improvement. A similar trend is observed in the larger Qwen3-1.7B (Q4_0) model, where the approximation increases TG performance by 29.0% (from 34.21 to 44.13 tokens/s). This disparity between phases can be attributed to the arithmetic intensity. In the memory-bound TG phase, the rigorous SiLU calculation—which typically involves expensive library calls or complex instruction sequences—stalls the pipeline. Our approximation replaces these with a compact sequence of simple vector arithmetic instructions, effectively hiding the computation latency behind memory access.

Crucially, this performance enhancement comes at a minimal cost to model accuracy. As shown in the “Degradation” column, the increase in PPL remains negligible for practical edge deployment. The degradation ranges from as low as 0.005% to a maximum of 1.739% (Qwen3-1.7B Q4_0). This confirms that our hardware-friendly approximation successfully exploits the resilience of LLMs

Table 2. Ablation Study of SiLU Approximation on Accuracy (PPL) and Performance (tokens/s)

Model	Method	Accuracy Metric		Performance (tok/s)	
		PPL ↓	Degradation	PP512 ↑	TG128 ↑
Qwen3-0.6B (Q4_0)	Scalar	23.97	–	82.93	35.18
	VectorWeaver	23.98	+0.005%	216.33	64.45
	VectorWeaver+SiLU	24.14	+0.671%	215.46	83.00
Qwen3-0.6B (Q8_0)	Scalar	21.85	–	65.04	35.00
	VectorWeaver	21.86	+0.021%	226.10	54.08
	VectorWeaver+SiLU	21.96	+0.487%	228.35	62.28
Qwen3-1.7B (Q4_0)	Scalar	18.97	–	35.98	17.54
	VectorWeaver	18.97	-0.007%	112.56	34.21
	VectorWeaver+SiLU	19.30	+1.739%	113.54	44.13
Qwen3-1.7B (Q8_0)	Scalar	17.10	–	25.90	16.44
	VectorWeaver	17.10	+0.020%	121.10	25.67
	VectorWeaver+SiLU	17.29	+1.103%	122.74	30.70

Scalar: Baseline implementation; **VectorWeaver:** Optimized implementation with intrinsics; **VectorWeaver+SiLU:** VectorWeaver with Mathematical Approximation. Metrics PP512 and TG128 represent prefill and decoding throughput respectively.

to minor precision loss, offering a pragmatic “knob” to trade negligible accuracy for substantial responsiveness in interactive scenarios.

5.6 Discussion

The comprehensive evaluation of VectorWeaver provides a multi-faceted validation of its core design principles and reveals critical insights into the optimization of LLM inference on RISC-V architectures. The empirical results, spanning micro-benchmarks, end-to-end throughput, scalability, and targeted ablation studies across both the TH1520 and SG2044 platforms, collectively build a compelling case for an automated, architecture-aware, and stage-specific approach. The significant performance gains are not the result of a single optimization, but rather the synergistic effect of the framework’s hierarchical methodology.

A foundational takeaway from our micro-benchmark analysis is the profound inadequacy of general-purpose optimization heuristics for highly specialized workloads like quantized LLM inference. The dramatic speedups observed in our “Vec-Dot” kernels—reaching up to 6.89× on SG2044 via the Width Expansion strategy—and in the “Vec-SiLU” function (up to 24.82×) through mathematical approximation, underscore the necessity of domain-specific knowledge. These transformations address subtle data-layout and computational bottlenecks that lie beyond the semantic reach of a standard compiler, which often fails to produce meaningful vectorization for quantized types. This motivates VectorWeaver’s multi-level code generation capability. While the automated search over RVV intrinsics delivers substantial gains, the ability to generate direct, fine-tuned assembly unlocks the hardware’s theoretical peak throughput. For instance, on the SG2044 platform, the assembly-optimized SiLU kernel reduces latency to 2.48 μ s, significantly outperforming the intrinsic version (4.46 μ s).

The empirical validation of our stage-aware architectural thesis is conclusive. The ablation study demonstrates that a “one-size-fits-all” kernel strategy is fundamentally flawed, particularly as hardware parallelism increases. On the 64-core SG2044, the static “PP-Best” configuration achieves high prefill throughput but suffers catastrophic performance degradation in the decoding phase due to synchronization and configuration overheads. Conversely, “TG-Best” excels at low-latency

decoding but fails to saturate the compute units during prefill. VectorWeaver’s dynamic dispatch mechanism successfully resolves this conflict, matching the peak performance of each specialized configuration without manual intervention. This result elevates stage-aware design from a heuristic improvement to a foundational requirement for high-performance RISC-V LLM inference.

Necessity of the Performance Model. Our experiments further highlight the critical role of the Performance Model in the optimization loop, driven by two key factors: validation cost and performance non-linearity. First, direct on-device validation in edge scenarios is prohibitively expensive. Cross-compilation environments separate build and execution, and rigorous end-to-end testing is time-consuming—for instance, a full perplexity (PPL) test for a 0.6B model using a scalar baseline on the TH1520 platform requires approximately 32 hours. This necessitates a reliable proxy model to rank candidates offline. Second, operator-level speedups do not linearly translate to end-to-end gains across different model scales. Our evaluation on the SG2044 reveals a complex behavior: while intrinsic-based optimizations provide gains for smaller models (e.g., Qwen3-0.6B and Gemma3-270m), they often degrade performance compared with scalar implementations in the Decode phase for larger models or other configurations. This counter-intuitive phenomenon suggests that instruction overheads and cache behaviors vary significantly with model size, making manual heuristic selection unreliable. Our performance model, trained on SG2044 data, achieves an average ranking accuracy of 75% when transferred to TH1520 tasks, demonstrating high utility as a robust filter against performance regressions. Crucially, the model allows us to identify platform-specific constraints, effectively pruning configurations that exceed the 32-register limit to prevent performance degradation caused by register spilling. This is particularly critical given the non-linear relationship observed between operator efficiency and model scale: on the SG2044, while intrinsic-based optimizations benefit smaller models, they often counter-intuitively degrade performance for larger models due to complex cache interactions.

Future Work. We observed a significant disparity in optimization efficacy across different model sizes, where the same kernel implementation behaves differently on 0.6B versus 4B models. This phenomenon presents an intriguing avenue for future research. These findings suggest that the interplay between micro-kernel instruction scheduling and macroscopic model architectural hyperparameters (e.g., hidden size, head count) is more tightly coupled than typically assumed. Furthermore, while our framework demonstrates adaptability across two distinct microarchitectures (SG2044 and TH1520), future work is needed to address zero-shot generalization across wider architectural variations, such as different VLEN configurations. While VectorWeaver currently addresses this via search, these insights indicate that future optimization frameworks for RISC-V may need to evolve from purely operator-centric views to more holistic, model-structure-aware cost functions to fully exploit hardware potential across varying scales.

6 Related Work

Our research is situated at the intersection of LLM inference optimization and automated code generation for novel hardware architectures. We survey the most relevant work in both domains.

6.1 LLM Inference Acceleration on CPUs

The pursuit of efficient LLM inference has spurred a wide range of optimization techniques. At a high level, methods like speculative decoding [24] and modifications to the attention mechanism, such as FlashAttention [13] and its variants [12, 40], aim at reducing the overall computational volume, though they are primarily designed and optimized for GPUs’ massively parallel architecture. Other orthogonal approaches focus on reducing the memory footprint and bandwidth requirements of the **Key-Value (KV)** cache [29], a primary bottleneck in the memory-bound decode stage, through techniques like compression and specialized data structures [2, 8, 15, 27, 28, 30].

Model quantization [22] has emerged as a critical strategy for deploying LLMs on resource-constrained devices, reducing model size and enabling faster integer-based computations. This is the area most adjacent to our work. While significant research has focused on quantization-aware training and post-training quantization to minimize accuracy loss, the challenge of generating high-performance kernels for these quantized data types on CPUs remains [9]. Most low-level CPU optimization work, exemplified by libraries like ggml in llama.cpp, has concentrated on architectures with established SIMD instruction sets [31], such as x86 (AVX) and ARM (NEON). RVV supports variable-length logical vector registers (via LMUL) critical for quantized kernels, a capability which AVX lacks. VectorWeaver distinguishes itself by being, to our knowledge, the first automated framework specifically designed to generate stage-aware, high-performance kernels for the emerging RVV extension [44]. It directly confronts the performance artifacts introduced by quantization schemes—such as memory discontinuities—which are often overlooked by higher-level optimization strategies.

6.2 Automated Kernel Tuning Frameworks

The automated generation of high-performance code for deep learning is a well-established field. Frameworks like Halide [38] pioneered the decoupling of algorithm logic from its execution schedule, allowing for powerful transformations. Subsequent systems like TVM [10] and its successor Ansor [47] introduced domain-specific languages and sophisticated search strategies, including machine learning-based cost models, to navigate the vast space of possible loop transformations [32] (tiling, fusion, reordering) for generating efficient code on diverse hardware backends like GPUs and CPUs. Unlike traditional frameworks (e.g., TVM, cuBLAS) that employ *size-based* kernel dispatch—selecting implementations based purely on problem dimensions ($M \times N \times K$) to optimize cache locality—VectorWeaver adopts a *stage-aware* approach. Stage-awareness accounts for *computational context* beyond data size: LLM inference’s Prefill and Decode stages differ fundamentally in arithmetic intensity, memory access patterns, and microarchitectural overhead (e.g., `vsetvli` latency, tail loop dominance), even when processing identical sequence lengths.

However, these frameworks typically operate at a higher level of abstraction. They primarily focus on optimizing loop nests and rely on a general-purpose backend compiler (e.g., LLVM [23]) for the final instruction selection and micro-architecture-specific code generation. This dependency can obscure low-level performance opportunities and bottlenecks. VectorWeaver differs from this paradigm in three fundamental aspects:

Direct Assembly Generation: Instead of relying on a compiler backend, VectorWeaver generates explicit RVV assembly. This provides fine-grained control over instruction scheduling, register allocation, and software pipelining, enabling it to exploit architectural nuances that are often inaccessible to a general-purpose compiler. **Domain-Specific Primitives:** It incorporates domain-specific optimization strategies, such as our proposed “Width Expansion,” which are designed to solve concrete problems that are invisible at the level of abstract loop transformations. **Stage-aware Specialization:** Its performance model and optimization workflow are explicitly designed to be stage-aware. This is a critical feature for the biphasic nature of LLM inference, which is not a primary consideration in general-purpose tuning frameworks that typically optimize for a single, static computational pattern.

7 Conclusion

This article introduced VectorWeaver, an automated framework designed to generate high-performance, stage-aware LLM inference kernels for the RISC-V architecture. We identified critical performance bottlenecks in existing frameworks stemming from quantization data layouts and the divergent computational profiles of the Prefill and Decode stages. To address these, we proposed

a novel “Width Expansion” vectorization strategy that leverages the unique architectural flexibility of the RVV extension. VectorWeaver automates the exploration of a complex optimization space and employs a stage-aware performance model to synthesize specialized kernels for each inference phase.

Our empirical evaluation on RISC-V hardware demonstrates that this automated, architecture-conscious approach yields substantial performance gains. VectorWeaver achieves speedups of up to 24x on critical micro-benchmarks and accelerates end-to-end llama.cpp throughput by up to 5.2x over leading baseline implementations. This work validates the immense potential of specialized, automated optimization for enabling high-performance AI on emerging open-source hardware. Future work will proceed along two primary axes: broadening the framework’s applicability by expanding operator support and targeting a wider range of RISC-V hardware with different vector lengths and pipeline characteristics, and deepening its intelligence by exploring more advanced search algorithms.

References

- [1] Ahmad Abdelfattah, Azzam Haidar, Stanimire Tomov, and Jack Dongarra. 2016. Performance, design, and autotuning of batched GEMM for GPUs. In *International Conference on High Performance Computing*. Springer, 21–38.
- [2] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J. Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. *Proceedings of Machine Learning and Systems* 6 (2024), 114–127.
- [3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient LLM inference by piggybacking decodes with chunked prefills. arXiv:2308.16369. Retrieved from <https://arxiv.org/abs/2308.16369>
- [4] Vicki H. Allan, Reese B. Jones, Randall M. Lee, and Stephen J. Allan. 1995. Software pipelining. *ACM Computing Surveys (CSUR)* 27, 3 (1995), 367–432.
- [5] Krste Asanović and David A. Patterson. 2014. Instruction sets should be free: The case for risc-v. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146* (2014).
- [6] Nick Brown. 2025. Is RISC-V ready for High Performance Computing? An evaluation of the Sophon SG2044. arXiv:2508.13840. Retrieved from <https://arxiv.org/abs/2508.13840>
- [7] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774* (2024).
- [8] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, et al. 2024. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069* (2024).
- [9] Longhao Chen, Yina Zhao, Qiangjun Xie, and Qinghua Sheng. 2024. Optimization of Armv9 architecture general large language model inference performance based on Llama. cpp. *arXiv preprint arXiv:2406.10816* (2024).
- [10] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.
- [11] Enfang Cui, Tianzheng Li, and Qian Wei. 2023. RISC-V instruction set architecture extensions: A survey. *IEEE Access* 11 (2023), 24696–24711.
- [12] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [13] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
- [14] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [15] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801* (2023).
- [16] ggml-org. 2025. llama.cpp: LLM inference in C/C++. Retrieved from <https://github.com/ggml-org/llama.cpp>
- [17] Samuel Greengard. 2020. Will RISC-V revolutionize computing? *Commun. ACM* 63, 5 (2020), 30–32.
- [18] Jude Haris, Rappy Saha, Wenhao Hu, and José Cano. 2024. Designing efficient LLM accelerators for edge devices. *arXiv preprint arXiv:2408.00462* (2024).

- [19] Carolina Imaniosky, Douglas A. Santos, Douglas R. Melo, Felipe Viel, and Luigi Dilillo. 2024. Special session: Reliability and performance evaluation of a RISC-V vector extension unit for vector multiplication. In *2024 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. IEEE, 1–6.
- [20] William Kahan. 1996. IEEE standard 754 for binary floating-point arithmetic. *Lecture Notes on the Status of IEEE 754*, 94720-1776 (1996), 11.
- [21] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. Pod-attention: Unlocking full prefill-decode overlap for faster llm inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 897–912.
- [22] Jiedong Lang, Zhehao Guo, and Shuyu Huang. 2024. A comprehensive study on quantization techniques for large language models. In *2024 4th International Conference on Artificial Intelligence, Robotics, and Communication (ICAIRC)*. IEEE, 224–231.
- [23] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004*. IEEE, 75–86.
- [24] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.
- [25] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. Llm inference serving: Survey of recent advances and opportunities. In *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–8.
- [26] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. 2019. Edge AI: On-demand accelerating deep neural network inference via edge computing. *IEEE Transactions on Wireless Communications* 19, 1 (2019), 447–457.
- [27] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. 2024. Minicache: Kv cache compression in depth dimension for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 139997–140031.
- [28] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. 2025. Clusterkv: Manipulating LLM kv cache in semantic space for recallable compression. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.
- [29] Yanyu Liu, Jingying Fu, Sixiang Liu, Yitian Zou, You Fu, Jiehan Zhou, and Shouhua Zhang. 2025. KV cache compression for inference efficiency in LLMs: A review. *arXiv preprint arXiv:2508.06297* (2025).
- [30] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [31] Mukul Lokhande, Gopal Raut, and Santosh Kumar Vishvakarma. 2025. Flex-PE: Flexible and SIMD multiprecision processing element for AI workloads. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 33, 6 (2025), 1610–1623.
- [32] Kathryn S. McKinley, Steve Carr, and Chau-Wen Tseng. 1996. Improving data locality with loop transformations. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 18, 4 (1996), 424–453.
- [33] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
- [34] Benjamin W. Mezger, Douglas A. Santos, Luigi Dilillo, Cesar A. Zeferino, and Douglas R. Melo. 2022. A survey of the RISC-V architecture software support. *IEEE Access* 10 (2022), 51394–51411.
- [35] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2025. A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology* 16, 5 (2025), 1–72.
- [36] ollama. 2025. Get up and running with OpenAI gpt-oss, DeepSeek-R1, Gemma 3 and other models. Retrieved from <https://github.com/ollama/ollama>
- [37] Vijay S. Pai, Parthasarathy Ranganathan, and Sarita V. Adve. 1997. The impact of instruction-level parallelism on multiprocessor performance and simulation methodology. In *Proceedings Third International Symposium on High-Performance Computer Architecture*. IEEE, 72–83.
- [38] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices* 48, 6 (2013), 519–530.
- [39] Michael Schlansker, Thomas M. Conte, James Dehnert, Kemal Ebcioglu, Jesse Zhixi Fang, and Carol L. Thompson. 1997. Compilers for instruction-level parallelism. *Computer* 30, 12 (1997), 63–69.
- [40] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2024), 68658–68685.
- [41] Raghubir Singh and Sukhpal Singh Gill. 2023. Edge AI: a survey. *Internet of Things and Cyber-Physical Systems* 3 (2023), 71–92.

- [42] James Stanier and Des Watson. 2013. Intermediate representations in imperative compilers: A survey. *ACM Computing Surveys (CSUR)* 45, 3 (2013), 1–27.
- [43] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [44] Pablo Vizcaino, Georgios Ieronymakis, Nikolaos Dimou, Vassilis Papaefstathiou, Jesus Labarta, and Filippo Mantovani. 2023. Short reasons for long vectors in HPC CPUs: A study based on RISC-V. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. 1543–1549.
- [45] Jianzhou Zhao, Santosh Nagarakatte, Milo M. K. Martin, and Steve Zdancewic. 2012. Formalizing the LLVM intermediate representation for verified program transformations. In *Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 427–440.
- [46] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* 1, 2 (2023).
- [47] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. 2020. Ansor: Generating {High-Performance} tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 863–879.
- [48] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.

Received 19 October 2025; revised 4 January 2026; accepted 17 February 2026